

Lab 08: Asp. Net MVC

ASP.NET MVC Application Manual with CRUD Operations

This manual explains the structure and functionality of an ASP.NET MVC application with full CRUD operations using the Entity Framework (Code-First approach). It aligns with the MVC learning path and includes a step-by-step explanation of each concept implemented in the application.

MVC Fundamentals

Setting up the Development Environment

1. **Install Visual Studio** (Community Edition or higher)
2. **Create New Project**
 - Choose: ASP.NET Web Application (.NET Framework)
 - Select: MVC Template
3. **Understand Folder Structure**
 - **Models/**: Holds business logic
 - **Views/**: Contains .cshtml files for UI
 - **Controllers/**: Contains controller classes

Installing Dependencies:

Go To Tools-> NuGet Package manager and type : **Microsoft. EntityFrameworkCore.SqlServer** and install it

Then type: **Microsoft. EntityFrameworkCore. Tools**

1.1 Models, Views, and Controllers (MVC)

ASP.NET MVC is a design pattern used for developing web applications. MVC stands for:

- **Model:** Represents the data and business logic.
- **View:** UI that displays data.
- **Controller:** Handles user input, works with the model, and returns a view.

Using Our Code:

- **Model:** Represents application data.
 - File: `student.cs` (Entity Model), `AddStudentViewModel.cs` (ViewModel)
- **View:** UI elements shown to users.
 - Files: `Add.cshtml`, `Edit.cshtml`, `List.cshtml`
- **Controller:** Handles requests and returns views or data.
 - File: `StudentController.cs`

Usage of MVC Architecture: MVC is used in modern web applications that require:

- Clear separation of concerns
- Test-driven development
- Scalable codebase
- RESTful APIs

1.2 Routing and URL Patterns

ASP.NET uses routes to map URLs to controller actions

- Default routing convention: `/Controller/Action/{id}`
- Examples:
 - `/Student/Add`
 - `/Student/List`
 - `/Student/Edit/{id}`

1.3 Action Methods and Parameters

- Action methods in the controller handle different types of HTTP requests (GET, POST).
 - Parameters such as `Guid id` are used for retrieving or manipulating specific data.
-

2. Working with Models

2.1 Creating and Binding Models

- `student.cs` defines the structure of student data.

```
public class student
{
    public Guid Id { get; set; }
    public string Name { get; set; }
    public string Email { get; set; }
    public string Phone { get; set; }
    public bool Subscribed { get; set; }
}
```

2.2 Entity Framework Integration

Entity Framework (EF) is an Object-Relational Mapper (ORM) that allows developers to work with a database using .NET objects. Instead of writing raw SQL queries, developers use EF to interact with the database in an object-oriented way.

- Uses **Code-First** approach.
- `ApplicationDbContext.cs` defines database context:
- `DbContext` is the core class that manages database operations. `DbSet<student>` represents the table in the database.
- In appsettings.json create a connection string
- In program.cs file inject DbContext telling which db to use

```
builder.Services.AddDbContext<ApplicationDbContext>(options =>
options.UseSqlServer(builder.Configuration.GetConnectionString("StudentPortal")));
```

-
- **Migrations:** EF tracks model changes via migrations

In package Manager run this command to reflect changes in the database :

Add-Migration InitialCreate

Then everytime a model updates and run

3. Views and Razor Syntax

Razor syntax is a markup syntax used in ASP.NET MVC (and ASP.NET Core) to seamlessly blend HTML with C# code in view files (`.cshtml`). It allows developers to write server-side code directly inside HTML, making it easy to create dynamic web pages.

- Using Razor helpers (Tag Helpers in ASP.NET Core)
- Help us to loop through the data

Example:

```
<form asp-action="Add" method="post">
  <input asp-for="Name" />
  <span asp-validation-for="Name"></span>
</form>
```

or

```
@if (Model.IsSubscribed)
{
  <p>Thank you for subscribing!</p>
}
else
{
  <p>You are not subscribed.</p>
}
```

3.1 Creating and Customizing Views

- Views are `.cshtml` files under `Views/Student/`
- Created for Add, Edit, and List actions.

3.2 Razor Syntax and Helpers

- Example inside `Add.cshtml`:

```
@model AddStudentViewModel
```

```
<form asp-action="Add" method="post">
  <input asp-for="Name" />
  <span asp-validation-for="Name"></span>
</form>
```

3.3 Partial Views and Layouts

- `_Layout.cshtml` provides a shared layout.
 - `_ValidationScriptsPartial.cshtml` handles validation scripts.
-

4. Controllers and Action Methods

4.1 Handling HTTP Requests

- `[HttpGet]` renders forms or views.
- `[HttpPost]` processes form submissions.

4.2 Action Filters and Result Types

- Return types: `View()`, `RedirectToAction()`
- Filters such as `[Authorize]` can be added for protection (not used here).

4.3 TempData, ViewData, and ViewBag

- **TempData:** Stores data until it's read (or for one redirect). Good for passing messages after a redirect (like success messages after **POST**).

```
public IActionResult Add(AddStudentViewModel viewModel)
{
    // Save to DB...
    TempData["Success"] = "Student added successfully!";
    return RedirectToAction("List");
}
```

- **ViewBag:** Used to pass data from controller to view.

```
public IActionResult Index()
{
    ViewBag.Message = "Welcome to the student portal!";
    return View();
}
```

In views: <h2>@ViewBag.Message</h2>

- **ViewData:** is a dictionary object (**ViewDataDictionary**) provided by **Controller**

It stores data as key-value pairs, where the key is a string and the value is an object.

```
public IActionResult Index()
{
    ViewData["Name"] = "Aisha";
    ViewData["Age"] = 23;
    return View();
}
```

5. Data Handling and CRUD Operations

5.1 Database Connection using Entity Framework

- Connection string in **appsettings.json**
- **ApplicationDbContext** manages database interactions.

5.2 CRUD Operations in StudentController

5.2.1 Create (Add)

[HttpGet]

```
public IActionResult Add() => View();
```

[HttpPost]

```
public async Task<IActionResult> Add(AddStudentViewModel viewModel)
{
    var student = new student
    {
        Name = viewModel.Name,
```

```

        Email = viewModel.Email,
        Phone = viewModel.Phone,
        Subscribed = viewModel.Subscribed
    };
    await dbContext.students.AddAsync(student);
    await dbContext.SaveChangesAsync();
    return View();
}

```

5.2.2 Read (List)

```

[HttpGet]
public async Task<IActionResult> List()
{
    var students = await dbContext.students.ToListAsync();
    return View(students);
}

```

5.2.3 Update (Edit)

```

[HttpGet]
public async Task<IActionResult> Edit(Guid id)
{
    var students = await dbContext.students.FindAsync(id);
    return View(students);
}

[HttpPost]
public async Task<IActionResult> Edit(student viewModel)
{
    var students = await dbContext.students.FindAsync(viewModel.Id);
    if (students is not null)
    {
        students.Name = viewModel.Name;
        students.Email = viewModel.Email;
        students.Phone = viewModel.Phone;
        students.Subscribed = viewModel.Subscribed;
        await dbContext.SaveChangesAsync();
    }
    return RedirectToAction("List", "Student");
}

```

5.2.4 Delete

```

[HttpPost]
public async Task<IActionResult> Delete(student viewModel)
{

```

```
var students = await dbContext.students.FindAsync(viewModel.Id);
if (students is not null)
{
    dbContext.students.Remove(students);
    await dbContext.SaveChangesAsync();
}
return RedirectToAction("List", "Student");
}
```

6. GET vs POST Methods Explained

GET Method

- Used to **retrieve** data.
- Examples:
 - `Add()` returns the empty form.
 - `List()` retrieves student records.
 - `Edit(id)` fetches student info.

POST Method

- Used to **submit** data to the server.
- Examples:
 - `Add(viewModel)` saves a new student.
 - `Edit(viewModel)` updates student details.
 - `Delete(viewModel)` removes a student.

7. REST APIs:

In an ASP.NET MVC or Web API application:

REST APIs are controller methods (also called endpoints) that respond to HTTP requests like GET, POST, PUT, DELETE.

These methods **represent RESTful operations** when they follow REST conventions, like operating on resources and using the correct HTTP verbs for different actions.

Example:

```
[HttpGet]
public IActionResult List() // RESTful 'GET' to fetch students
{
    // Get all students
}

[HttpPost]
public IActionResult Add(AddStudentViewModel viewModel) // RESTful 'POST' to create student
{
    // Add a new student
}
```